Selecting Reliable LLM Reasoning

DASE7506 Mini-project 2 | Final method v11 | 29 October 2026

Yu Qinghao | Student ID: 3036843599

1. Question and main finding

Can explicit arithmetic evidence improve a selector that already compares candidate answers using fixed embeddings? The hypothesis is that local equality checks and propagation of arithmetic slips add complementary evidence. The final method combines a linear answer-group scorer with a small learned correction. It selects one of the four supplied candidates and performs no language-model generation.

The frozen final model obtains 397/467 = 85.01070663811564% on the complete official test bank. Relative to the earlier v3 anchor (390/467), this is seven additional correct answers, or 1.50 percentage points. The combined arithmetic correction also improves three-fold train-question cross-validation from 1,740 to 1,764 correct out of 2,114, and validation from 746 to 751 out of 798. These results support the hypothesis on this bank, subject to the development-history limitations below.

2. Data and evaluation protocol

The supplied dataset contains 4,755 training, 798 validation and 467 test questions, each with four candidate solutions and 1,024-dimensional fixed embeddings. The official numeric parser groups equivalent fractions and decimals. The scorer awards correctness for the selected final number; it does not verify every reasoning step.

There are 2,114 conflicting-answer training questions and 355 conflicting-answer validation questions. The remaining 443 validation questions have a single valid answer group. Every test question has conflicting valid answers. Consequently, overall validation accuracy is easier than test accuracy; the relevant final validation comparison is 308/355 = 86.76% on conflicting questions.

All fitting uses supplied training inputs and references. The final correction family, regularization and strength are selected with three shuffled question-level folds (seed 7506). Each fold refits the anchor, its normalization, correction normalization and correction weights using only that fold's training questions. A declared validation retention threshold of at least 750/798 is then applied. The selected v11 source and weights were hashed before one official v11 test evaluation.

3. Development chronology and disclosure

Earlier aggregate test scores for v1, v2, v3, v9 and v10 were already known before v11. Public repository README scores were also read. No per-question test errors were inspected, and test references were not used to fit or choose v11 parameters. Nevertheless, the project has repeated development and aggregate test exposure; it is not an untouched blind-test study. The final CV grid is itself used for selection, rather than being a nested unbiased estimate. An instructor audit bank remains the stronger generalization check.

4. Architecture and inference

Stage A: answer-group anchor. Parse the four final answers, merge numerically equivalent answers, and represent each group with 1,112 features: mean embedding (1,024), four support/dispersion descriptors, means and ranges of 39 surface/similarity features (78), and six numeric plausibility descriptors. A scalar linear score is shared across groups.

The anchor is trained on all nonempty subsets within each same-answer group. Correct and incorrect subsets form preference pairs. Each informative question receives total pair weight one. A weighted logistic pair loss with L2 coefficient 0.01 is minimized using L-BFGS-B. Fold-local statistics are fitted on original full groups. At inference only the supplied full groups are scored; augmentation is a training procedure.

Stage B: local arithmetic evidence (48 coordinates). A bounded parser normalizes common numeric notation and interprets a whitelist of arithmetic AST nodes using exact fractions. It checks numeric equalities, mismatch rates, relative residuals and agreement of the last calculation with final answers. Sixteen candidate descriptors are pooled by mean, minimum and maximum inside each answer group. Unsupported symbolic or ambiguous expressions contribute no checked equality.

Stage C: propagated repair evidence (46 coordinates). Traverse extracted equalities in textual order. When a reported numeric result disagrees with its recomputation, record a proposed substitution. Later left-hand numeric expressions are recomputed with earlier substitutions. Conflicting replacements are marked ambiguous. Fourteen candidate descriptors summarize changes and final-answer agreement, pooled by mean/minimum/maximum; four additional descriptors summarize support from the group's own and other candidates' corrected endpoints.

Example trace	Observed value	Recomputed value
2 * 3 = 5	5	6
5 * 4 = 20	20	24 after substituting 5 with 6

This repair is evidence, not a semantic proof: a repeated numeral can denote a different quantity. The learned correction can discount unreliable features. Processing is bounded to 20,000 text characters and 48 extracted equalities; expressions allow at most 128 characters and 48 AST nodes, with rational magnitude limits. No eval, external API, pretrained weights or answer lookup is used.

Final score. $s(g) = w^T x(g) + v^T f(g)$, where f contains 48 + 46 = 94 arithmetic/repair features. The anchor is fixed while the correction is fitted on original full-group preference pairs. Its loss is weighted logistic loss on the anchor margin plus correction margin, with alpha/2 times squared coefficient norm. Selected alpha = 0.1 and correction strength = 1.0. Train-fitted feature scaling is absorbed into the saved coefficients. The highest-scoring group wins; a deterministic text-based tie-break selects one member.

5. Ablations and model selection

The final comparison includes the unchanged anchor and three correction variants: local arithmetic only, propagation only, and their combination. Each correction checks alpha in {0.01, 0.1, 1.0} and strength in {0.25, 0.5, 1.0}: 27 settings plus the baseline. The table gives each family's best selection-CV result on the same 2,114 questions. Differences are not independent significance tests.

Family	CV correct	CV accuracy
Anchor only	1,740	82.31%
Local arithmetic only	1,759	83.21%
Propagation only	1,750	82.78%
Combined (selected)	1,764	83.44%

Question fold	Anchor correct	Combined correct	Net change
Fold 1	588	591	+3
Fold 2	584	594	+10
Fold 3	568	579	+11

The combined correction improves every fold and adds five successes over local arithmetic alone. Propagation by itself is weaker than local arithmetic, so its value is complementary rather than sufficient. These results motivate a small correction layer rather than replacing the anchor with a large neural scorer.

6. Failed alternatives and validation evidence

Experiment family	Best validation correct / 798
Original v3 anchor	746
Nonlinear antisymmetric pair comparison	746 (baseline retained)
PCA / whitening of group embeddings	745
Alternative linear ranking losses	747
Tree residual ensemble	749
Neural residual ensemble (v9)	752
Local arithmetic correction (v10)	750
CV-selected combined correction (v11)	751

The v4-v9 investigation evaluated numerous architectures and blends on the reused validation bank. Its chosen neural model improved validation to 752/798 but scored 384/467 on test, below v3. This is direct evidence that validation improvements were not reliably predictive of generalization in this project. V10 then used a small arithmetic correction, reaching 394/467. V11 adds propagation and uses question-fold selection with refitted anchors plus a validation retention check. Its improvement should still be interpreted with the earlier exposure disclosed on page 1.

V11 training and CV took 27.59 seconds locally, excluding dependency installation and earlier prerequisite searches. Earlier recorded costs include 53.66 seconds for the v3 search and 26.01 seconds for the v5 pair-cache/PCA experiment. The exploratory neural search took 1,317.48 seconds; tree search 76.94 seconds. These experiments are recorded rather than presented as part of final inference.

7. Official results, reliability and resources

Frozen method	Validation	Complete official test
v3: group anchor	746/798 (93.48%)	390/467 (83.51%)
v9: neural residual	752/798 (94.24%)	384/467 (82.23%)
v10: local arithmetic	750/798 (93.98%)	394/467 (84.37%)
v11: combined arithmetic/repair	751/798 (94.11%)	397/467 (85.0107%)

V11 makes 70 test selection errors. Error causes on test were not inspected. The aggregate gain over v10 is three questions (0.64 percentage points), and the final score exceeds 85% by only 0.0107 percentage points. This is a narrow benchmark improvement, not evidence that the selector is universally reliable. We have not calibrated score margins into correctness probabilities.

Resource / verification	Recorded result
Final inference arrays	1,112 anchor + 94 correction coefficients; float64
Final checkpoint file	10,166 bytes, uncompressed NPZ
Test prediction + independent scoring	1.504 seconds, fresh serial processes
Local hardware / runtime	Intel Core i9-14900HX; Windows; Python 3.12
Numerical thread limit	4 per process; predictor and scorer run serially
Synthetic repair checks	Passed, including 2*3=5 followed by 5*4=20
Permutation and batch checks	Passed on 64 validation inputs, no reference labels

Only model.npz is loaded as an inference asset. The original v3 checkpoint is also retained for retraining and historical verification; counting its 28,205 bytes conservatively with the final checkpoint gives 38,371 bytes, still far below 16 MiB. Source files and ordinary library packages are excluded under the assignment rules. Timing includes process startup, imports, input/checkpoint loading, preprocessing and official scoring. Reference Xeon Platinum 8457C runtime has not been measured; local results do not certify that machine's eligibility.

8. Limitations and checks

The arithmetic parser covers explicit bounded numerical expressions, not algebra, unit reasoning or all natural-language calculations. It can miss malformed or unusual notation. Propagation uses numerical identity, which can confuse quantities with equal values; repeated mismatches and ambiguous substitutions can weaken evidence. Consistent arithmetic can still implement the wrong interpretation of a question. Correct final answers can also follow incorrect intermediate reasoning.

Candidate permutation and batch invariance checks reduce the risk of positional shortcuts and fixed-bank assumptions. The selector accepts caller-provided inputs and does not read reference files. The official data hashes, numeric parser and scoring rules remain unchanged. The frozen asset/source SHA-256 manifest identifies the evaluated implementation, but hashing does not establish statistical independence.

9. Reproduce the frozen result

Use Python 3.12. Install the pinned requirements in code/requirements.txt (NumPy 2.5.3, SciPy 1.18.1, scikit-learn 1.9.1, joblib 1.6.0). From the repository root:

```
cd code

py -3.12 -m pip install -r requirements.txt

py -3.12 fetch_data.py

py -3.12 check_data.py

py -3.12 evaluate_v11.py

py -3.12 verify_v11.py
```

On Linux/macOS replace py -3.12 with the Python 3.12 executable. The data fetcher uses the official pinned upstream data commit and validates hashes. After setup, inference works offline. evaluate_v11.py checks runs/experiment_v11/freeze.json and runs the unchanged official predictor and scorer in fresh processes for validation and test. Expected results are 751/798 and 397/467. No retraining is needed. The prediction entry is repair_v11.py and the checkpoint is runs/experiment_v11/model.npz.

10. Reproduce training

```
py -3.12 train_augmented_ranker.py

py -3.12 verify_anchor_rebuild.py

py -3.12 train_v5.py

py -3.12 train_v11.py
```

The first command reconstructs the v3 anchor into runs/augmented_ranker; the next verifies its numerical equivalence to the shipped anchor. train_v5.py regenerates the train-only subset-pair cache at runs/experiment_v5/train_pairs.npz (and historical PCA comparisons). train_v11.py uses that cache with explicit question boundaries to refit fold anchors and residuals, then fits the selected correction on all informative train questions. It writes runs/experiment_v11 and does not open test references. Run in a separate checkout to preserve the distributed frozen files. See RESULTS_V11.md for the evidence index and dependency details.

11. Attribution and substantive AI assistance

General answer-group pooling, within-answer subset augmentation and equal-question weighting were informed by public repositories [1,2]. Local arithmetic checks and a small residual layer were informed by [1]; cross-candidate arithmetic evidence and repair ideas were discussed in [3]. The 94-feature combined correction and its integration/CV pipeline were independently implemented here. No third-party source file, checkpoint, prediction file, external dataset or additional pretrained weight was copied into the method. Reading these sources influenced design, so conceptual reuse is explicitly acknowledged.

[1] CaspianW/dase7506-mp2. https://github.com/CaspianW/dase7506-mp2 . Arithmetic study commit: 6709b9ff3cfaea1fed0a4f897f6796358ccfcd02.

[2] vectorBH6/DASE7506-MP2. https://github.com/vectorBH6/DASE7506-MP2 . Group supervision and fold-local normalization.

[3] Lyan9145/DASE7506-MP2. https://github.com/Lyan9145/DASE7506-MP2 . DeepSets and cross-candidate arithmetic/repair ideas.

[4] Official course data and evaluation. https://github.com/xudongwu-0/DASE7506-MP2 . Pinned data commit bb801f4f2e7629d270df765ab12c333b2dc616bb.

OpenAI Codex substantially assisted public-source research, method design, implementation, experiment execution, verification, report writing and submission preparation. The student remains responsible for understanding the work. PUBLIC_METHODS.md, runs/v10-sources.json and runs/experiment_v11/NOTES.json retain provenance and chronology. The course/GSM8K license and attribution notices remain in code/data/.